

Chapter Overview

Chapter 15 covers microservices architecture as applied to ShopVerse: service decomposition, inter-service communication (REST, gRPC, Kafka), service discovery, OpenFeign, distributed tracing, API versioning, data ownership, BFF pattern, service mesh, and testing strategies (contract testing, integration testing).

Sub-Chapter	Topic
15-01	Microservices vs Monolith – when to split
15-02	ShopVerse Service Decomposition – domain-driven boundaries
15-03	Synchronous Communication – REST via OpenFeign, gRPC
15-04	Asynchronous Communication – Kafka events between services
15-05	Service Discovery – Kubernetes DNS, Spring Cloud, Eureka
15-06	Distributed Tracing – OpenTelemetry, Zipkin, Grafana Tempo
15-07	API Versioning – URI, header, content negotiation
15-08	Data Ownership – database per service, no shared DB
15-09	Backend for Frontend (BFF) Pattern
15-10	Service Mesh – Istio, sidecar proxy, mTLS
15-11	Contract Testing – Pact, Spring Cloud Contract
15-12	Integration & E2E Testing – TestContainers

Aspect	Monolith	Microservices
Deployment	Single artifact; deploy all or nothing	Each service deploys independently
Scalability	Scale all or nothing	Scale individual services by load
Failure isolation	One bug can crash all features	Failures isolated to one service
Technology diversity	Single stack	Each service can use best-fit tech
Operational complexity	Low (one process)	High (service mesh, tracing, CD pipelines per service)
Data consistency	ACID transactions	Eventual consistency; saga patterns needed
Team structure	One team; coordination overhead	Conway's Law: small autonomous teams per service

Decision Framework

When to split from Monolith

Team size > 8 engineers – coordination overhead outweighs cohesion benefits

Independent release cadence – one module deploys 10×/day; another 1×/month

Vastly different scaling needs – search scaled 100× vs order 5×

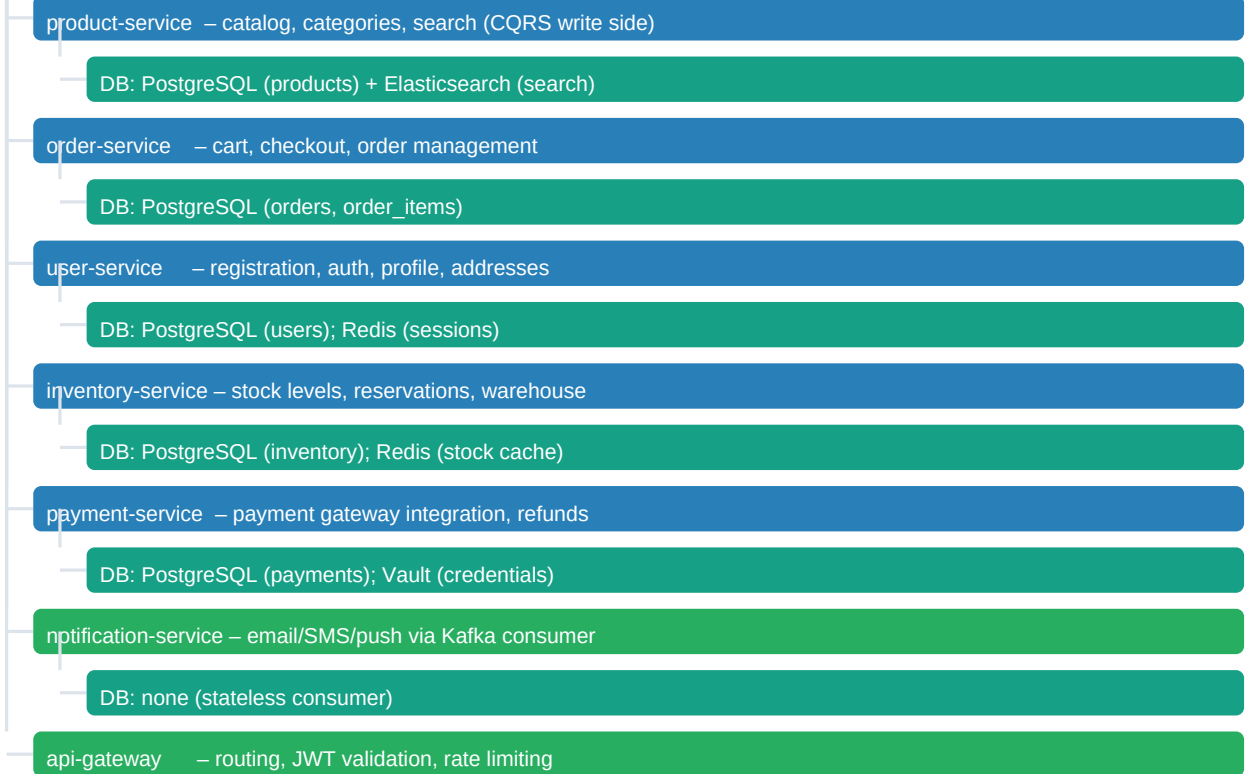
Tech stack mismatch – ML model in Python; rest in Java

When NOT to split

Startup phase – premature split = distributed monolith + complexity

Team size < 5 – microservices overhead exceeds benefit

ShopVerse Microservices (Domain-Driven Design)



Service	Port	Key Events Published	Consumed Events
product-service	8080	ProductCreated, PriceUpdated	OrderPlaced (update salesCount)
order-service	8081	OrderPlaced, OrderCancelled	StockReserved, PaymentSucceeded
inventory-service	8082	StockReserved, StockReleased	OrderPlaced, OrderCancelled
payment-service	8083	PaymentSucceeded, PaymentFailed	StockReserved

```

@FeignClient(name = "inventory-service",
url = "${services.inventory.url}",
configuration = FeignConfig.class)
public interface InventoryClient {
@GetMapping("/api/v1/inventory/{productId}/stock")
int getStock(@PathVariable Long productId);
@PostMapping("/api/v1/inventory/{productId}/reserve")
ReservationResponse reserve(@PathVariable Long productId,
@RequestBody ReserveRequest request);
}
// FeignConfig: retry + circuit breaker
@Bean public Retryer feignRetryer() {
return new Retryer.Default(100, 1000, 3); // 3 retries, 100ms-1s backoff
}

```

	REST (OpenFeign)	gRPC
Protocol	HTTP/1.1 or HTTP/2	HTTP/2 (always)
Payload	JSON (human-readable)	Protobuf (binary, 5-10× smaller)
Code generation	Manual DTO classes	Proto schema → generated stubs
Streaming	No native	Bidirectional streaming supported
Browser-friendly	Yes	No (needs gRPC-Web proxy)
ShopVerse use	Service-to-service REST	Inventory stock check (high frequency, latency sensitive)

Event Schema (Avro with Schema Registry)

```
{
  "namespace": "com.shopverse.events",
  "type": "record",
  "name": "OrderPlacedEvent",
  "fields": [
    { "name": "orderId", "type": "long" },
    { "name": "customerId", "type": "long" },
    { "name": "productId", "type": "long" },
    { "name": "quantity", "type": "int" },
    { "name": "totalAmount", "type": "double" },
    { "name": "placedAt", "type": "string" }
  ]
}

// Producer
@Service public class OrderEventPublisher {
  private final KafkaTemplate<String, OrderPlacedEvent> kafka;
  public void publishOrderPlaced(Order order) {
    String key = String.valueOf(order.getCustomerId()); // partition by customer
    kafka.send("orders.placed", key, mapToEvent(order));
  }
}
```

Kafka Setting	Value	Reason
acks	all	All ISR must acknowledge; no data loss
enable.idempotence	true	Exactly-once per partition; no duplicates
Partitioning key	customerId	All orders for same customer in same partition = ordered delivery

Discovery Type	Mechanism	ShopVerse Approach
DNS-based (Kubernetes)	Service name resolves to ClusterIP	Inventory-service:8082 inside K8s cluster
Client-side (Eureka)	Client queries registry; load-balancing	Spring Cloud Netflix Eureka (non-K8s)
Server-side (K8s Service)	kube-proxy routes to healthy pods	Default in K8s; no client code needed

```
# Kubernetes Service - DNS-based discovery
apiVersion: v1
kind: Service
metadata:
  name: inventory-service
  namespace: shopverse
spec:
  selector:
    app: inventory
  ports:
    - port: 8082
  targetPort: 8082
# Other pods reach it via: http://inventory-service:8082
# OpenFeign: ${services.inventory.url}=http://inventory-service:8082
```

Trace Propagation

Incoming HTTP Request: POST /api/v1/orders

API Gateway – creates TraceId=abc123; SpanId=0001; injects B3 headers

X-B3-TraceId: abc123, X-B3-SpanId: 0001 passed downstream

order-service – SpanId=0002; records span start/end time

Calls inventory-service (SpanId=0003, parentSpan=0002)

inventory-service – SpanId=0003; checks Redis cache + Postgres

All spans exported to Zipkin/Grafana Tempo with TraceId=abc123

Full waterfall view: gateway(5ms)+order(120ms)+inventory(30ms)=155ms

```
# Spring Boot 3 + Micrometer Tracing
implementation "io.micrometer:micrometer-tracing-bridge-otel"
implementation "io.opentelemetry.instrumentation:opentelemetry-spring-boot-starter"
# application.yml
management.tracing.sampling.probability: 1.0 # 100% in dev; 0.1 in prod
otel.exporter.otlp.endpoint: http://tempo:4317 # Grafana Tempo
```

Strategy	Example	Pros	Cons
URI versioning	/api/v1/products, /api/v2/products	Simple; cacheable; easy routing	URI pollution; URI should identify resource not version
Header versioning	API-Version: 2	Clean URIs; invisible to client	Not cacheable by default; hidden from URL
Content negotiation	Accept: application/vnd.shopverse.v2+json	Standards-based (RFC 5988)	Complex; tooling support limited

```
// URI versioning - ShopVerse standard
@RestController @RequestMapping("/api/v1/products")
public class ProductControllerV1 { ... }
@RestController @RequestMapping("/api/v2/products")
public class ProductControllerV2 {
// V2 adds facets, pagination cursor, enhanced search response
}
// Deprecation policy:
// 1. Announce deprecation in response headers: Sunset: Sat, 01 Jan 2027 00:00:00 GMT
// 2. Support v1 and v2 simultaneously for 12 months
// 3. Remove v1 only after all clients migrated (tracked via API analytics)
```

Versioning Decision	Rule
Breaking change	New version: add /v2/ endpoint; deprecate /v1/
Non-breaking (additive)	Same version: add optional field; backward compatible
Deprecation notice	Sunset header + Deprecation header; 12-month runway

Anti-Pattern: Shared Database

WRONG: Multiple services share same DB schema

order-service reads products table directly (couples services)

Schema change breaks all services simultaneously

No independent scaling of each service's data needs

CORRECT: Database per Service

order-service owns: orders, order_items, carts

product-service owns: products, categories, product_translations

Cross-service reads: via REST API call or Kafka event

order-service caches product name/price at order creation time (point-in-time)

Never queries product-service DB directly

Pattern	When	Implementation
API composition	Read current data from multiple services	Service aggregates product + inventory + reviews
Event-sourced snapshot	Preserve state at event time	Order stores product name+price at order creation
CQRS read model	Denormalised cross-service read	order-service keeps local product summary from Kafka

BFF Architecture

Clients – different needs, different form factors

Mobile App BFF – compact payload, fewer fields, push-optimised

Aggregates: product summary + stock (2 fields) + price

Reduces payload: 5 KB vs 20 KB generic response

Web App BFF – rich payload, SEO metadata, full product detail

Aggregates: product detail + reviews + recommendations + breadcrumb

Partner API BFF – bulk operations, webhook support, OAuth2 client credentials

Each BFF calls downstream microservices and aggregates

product-service, inventory-service, review-service, recommendation-service

```
@RestController @RequestMapping("/mobile/v1")
public class MobileBff {
    private final ProductClient productClient;
    private final InventoryClient inventoryClient;
    @GetMapping("/products/{id}")
    public MobileProductDto getProduct(@PathVariable Long id) {
        CompletableFuture<ProductDto> p = CompletableFuture
            .supplyAsync(() -> productClient.findById(id));
        CompletableFuture<Integer> s = CompletableFuture
            .supplyAsync(() -> inventoryClient.getStock(id));
        return new MobileProductDto(p.join(), s.join());
    }
}
```

Concern	Without Mesh	With Istio Sidecar
mTLS between services	Manual cert management in each service	Automatic, Envoy sidecar handles certs
Load balancing	kube-proxy (L4)	Envoy L7 (HTTP/2, gRPC-aware, retry)
Circuit breaker	Resilience4j in each service	Istio DestinationRule outlier detection
Distributed tracing	Micrometer in each service	Envoy generates spans automatically
Traffic shifting (canary)	Manual; requires code change	VirtualService: 90% stable / 10% canary

```
# Istio VirtualService - canary release (10% traffic to v2)
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata: { name: product-service }
spec:
  hosts: [product-service]
  http:
  - route:
    - destination: { host: product-service, subset: v1 }
    weight: 90
    - destination: { host: product-service, subset: v2 }
    weight: 10
```

```
// Consumer side (order-service tests)
@ExtendWith(PactConsumerTestExt.class)
@PactTestFor(providerName = "inventory-service")
class InventoryClientContractTest {
    @Pact(consumer = "order-service")
    public RequestResponsePact stockPact(PactDslWithProvider b) {
        return b
            .given("Product 1 has stock")
            .uponReceiving("GET stock for product 1")
            .path("/api/v1/inventory/1/stock").method("GET")
            .willRespondWith().status(200)
            .body(newJsonBody(o -> o.integerType("stock", 10)).build())
            .toPact();
    }
    @Test @PactTestFor(pactMethod = "stockPact")
    void testGetStock(MockServer mock) {
        InventoryClient client = new InventoryClient(mock.getUrl());
        assertThat(client.getStock(1L)).isEqualTo(10);
    }
}
```

Test Type	Scope	Tools
Unit test	Single class; mock dependencies	Unit 5, Mockito
Contract test	Consumer-provider API contract	Pact, Spring Cloud Contract
Integration test	Service + real DB + real Redis	TestContainers, @SpringBootTest
E2E test	Full flow across all services	Cucumber, RestAssured, TestContainers Compose

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers
class OrderServiceIntegrationTest {
    @Container static PostgreSQLContainer<?> pg =
        new PostgreSQLContainer<>("postgres:16")
            .withDatabaseName("shopverse_test");
    @Container static GenericContainer<?> redis =
        new GenericContainer<>("redis:7").withExposedPorts(6379);
    @Container static KafkaContainer kafka =
        new KafkaContainer(DockerImageName.parse("confluentinc/cp-kafka:7.6"));
    @DynamicPropertySource static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", pg::getJdbcUrl);
        r.add("spring.data.redis.host", redis::getHost);
        r.add("spring.data.redis.port", () -> redis.getMappedPort(6379));
        r.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
    }
    @Test void placeOrderE2E() { ... }
}
```

■ Do

- ✓ Give each microservice its own database – no shared schema; services communicate via API or events.
- ✓ Use Kafka for async cross-service events; OpenFeign for synchronous service calls with circuit breaker.
- ✓ Implement contract tests (Pact) to catch API breaking changes before deployment.
- ✓ Use DNS-based discovery in Kubernetes – service name resolves to ClusterIP automatically.
- ✓ Propagate TraceId across all service calls (OpenTelemetry) – enables end-to-end trace waterfall.
- ✓ Version APIs (/v1, /v2); announce deprecation with Sunset header; support old version 12 months.
- ✓ Use BFF pattern for mobile and web – tailor aggregation and payload size per client type.
- ✓ Use TestContainers for integration tests – real Postgres, Redis, Kafka; no mocking infra.
- ✓ Implement Saga pattern with compensating transactions for multi-service order workflows.
- ✓ Use Istio VirtualService for canary releases – 10% to new version before full rollout.

■ Anti-Patterns

- Shared database between services – tight coupling; schema change breaks all services simultaneously.
- Synchronous call chains across 5+ services – one slow service blocks the entire chain.
- Distributed monolith – microservices that deploy together and share a DB; worst of both worlds.
- No contract tests – service A changes API; service B breaks in production.
- No distributed tracing – impossible to debug which service caused 500ms latency.
- Breaking changes without versioning – old clients fail when API changes without /v2 endpoint.
- Premature microservices – 2-person team with 10 services; ops overhead > business benefit.

Sub-Chapter	Key Point	One-Liner
15-01 vs Monolith	Split when team > 8	Independent deploy cadence = split signal
15-02 Decompose	6 services + gateway	product, order, user, inventory, payment, notification
15-03 Sync Calls	OpenFeign + Retry	@FeignClient; 3 retries; circuit breaker
15-04 Async Calls	Kafka + Avro + Schema Registry	OrderPlacedEvent partitioned by customerId
15-05 Discovery	K8s DNS	inventory-service:8082 inside cluster namespace
15-06 Tracing	OpenTelemetry + Tempo	TraceId propagated via B3 headers
15-07 Versioning	URI /v1 /v2	Sunset header; 12-month deprecation window
15-08 Data Ownership	DB per service	No cross-service DB reads; cache locally or call API
15-09 BFF	Mobile BFF + Web BFF	Aggregate for form factor; reduce payload
15-10 Service Mesh	Istio sidecar	mTLS, canary routing, L7 LB automatically
15-11 Contract testing	Pact on consumer side	Provider verifies Pact file; catch breaks before prod
15-12 Testcontainers	Real infra in tests	Postgres + Redis + Kafka in Docker for integration

Interview Quick-Check

Question	Concise Answer
Why database per service?	Decoupling: service A's schema change doesn't break service B. Independent scaling. Service owns its data.
OpenFeign vs Kafka?	OpenFeign: synchronous, request-response, use when caller needs immediate result (stock check). Kafka: asynchronous, event-driven, use for decoupling and replayability.
What is a Saga?	Sequence of local transactions per service. Each step has a compensating transaction. On failure, call compensating transaction to undo.
Contract testing?	Consumer defines expected API interactions (Pact). Provider verifies against Pact file. Catches breaking changes before prod.
Distributed tracing?	TraceId generated at gateway; propagated via B3/W3C headers. Each service creates a span. Exporter sends to collector.